POLARIS
SCHOOL OF TECHNOLOGY

Text → Tokens → Vectors

Byte Pair Encoding, WordPiece, embedding layers, and positional embeddings

These notes explain how a Transformer converts raw text into an ordered sequence of vectors. They include worked examples, mathematical intuition, practical code, exercises, and a self-check answer key.

Name:

Roll number:

Course:

Date:

1. The big picture

Learning outcomes

After studying these notes, you should be able to:

- explain why modern NLP systems use subword tokens;
- perform several BPE merges by hand;
- contrast BPE and WordPiece training intuitions;
- trace tensor shapes through an embedding layer;
- calculate embedding parameters and storage; and
- explain why a Transformer needs positional information.

A language model does not directly receive words or characters. It receives integer IDs, then converts those IDs into vectors. Because attention alone has no built-in sense of sequence order, the model must also receive positional information.

Stage 1

Tokenization

`"playing!"`

→ `["play", "##ing", "!"]`

→ `[27, 91, 8]`



Stage 2

Embedding

`[27, 91, 8]`

→ rows 27, 91, 8

→ matrix $X \in \mathbb{R}^{3 \times d}$



Stage 3

Position

token vector

+ position vector

→ ordered input

Why this matters

Tokenization affects model cost, latency, context-window usage, handling of typos and rare names, source-code quality, and multilingual fairness. The embedding table can contain millions of parameters. Positional design determines how the model represents order and distance.

1.1 Keep two learning processes separate

1. **Tokenizer training** learns a vocabulary and segmentation rules from a text corpus. It normally happens before neural-network training.
2. **Model training** uses backpropagation to learn embedding vectors and all other neural-network parameters.

Applying a finished tokenizer is usually deterministic. The tokenizer does not run gradient descent each time it reads a sentence.

2. Why tokenize into subwords?

A finite system must represent an effectively unlimited set of strings: new words, names, spelling mistakes, code identifiers, numbers, URLs, and many writing systems. The choice of basic unit creates an engineering trade-off.

Basic unit	Vocabulary size	Sequence length	Main advantage	Main problem
Whole words	Very large	Short	Natural-looking units	Unknown words, typos, and rare forms
Characters / bytes	Very small	Long	Almost any string is representable	More positions and more computation
Subwords	Medium	Medium	Reusable pieces with fewer unknowns	Segmentation quality varies by domain/language

Subword intuition

Common words may stay whole

`playing → playing`

`engineer → engineer`

Rare words reuse smaller pieces

`unplayable → un + play + able`

`playbot → play + b + ot`

$$\text{Embedding parameters} = V \times d$$

$$\text{Approximate self-attention work per layer} \propto n^2 d$$

Here V is vocabulary size, d is embedding width, and n is sequence length. Increasing V may shorten sequences but enlarges the embedding table. Decreasing V may save parameters but lengthen sequences. If token count doubles, the n^2 part of attention work grows by roughly 4×.

Quick check 1

For each application, which starting point seems more suitable: words, subwords, characters, or bytes? Give one reason.

1. A spell-checker that must process typing errors
2. A model for source code with new variable names
3. A small classifier for a fixed list of 20 commands

3. Byte Pair Encoding (BPE)

3.1 Core idea

BPE begins with small symbols—commonly characters or bytes—and repeatedly merges the most frequent adjacent pair. Each merge creates a new vocabulary item. It can be understood as a greedy compression procedure: repeated patterns receive shorter representations.

$$(a^*, b^*) = \arg\max_{(a,b)} \text{count}(a,b)$$

$$\text{vocabulary} \leftarrow \text{vocabulary} \cup \{a^*b^*\}$$

$$\text{corpus} \leftarrow \text{replace every adjacent } (a^*,b^*) \text{ with } a^*b^*$$

BPE training procedure

1. Split each word into initial symbols and optionally add an end-of-word marker.
2. Count every adjacent pair, weighted by word frequency.
3. Select the pair with the highest count.
4. Merge every occurrence of that pair.
5. Repeat until reaching the vocabulary or merge budget.
6. Store the base symbols and ordered merge rules.

```
function TRAIN_BPE(corpus, K):
    represent every word as a list of symbols
```

```
    merges = []

    repeat K times:
        counts = weighted counts of adjacent symbol pairs
        best_pair = pair with maximum count
        replace best_pair everywhere with its concatenation
        append best_pair to merges

    return vocabulary, merges
```

Do not assume linguistic meaning

A frequent BPE piece may match a prefix, suffix, root, punctuation pattern, or an arbitrary fragment. BPE optimizes a frequency-based compression heuristic, not a dictionary of morphemes.

3.2 Worked BPE example

Suppose the training corpus contains:

Word	Frequency	Initial symbols
low	5	`l o w </w>`
lower	2	`l o w e r </w>`
newest	6	`n e w e s t </w>`
widest	3	`w i d e s t </w>`

Some initial weighted pair counts are:

Pair	Weighted count	Reason
`(e,s)`	9	newest ×6 + widest ×3
`(s,t)`	9	newest ×6 + widest ×3

`(t,</w>)`	9	newest ×6 + widest ×3
`(w,e)`	8	lower ×2 + newest ×6
`(l,o)`	7	low ×5 + lower ×2

Several pairs tie for highest count. A real implementation applies a deterministic tie-breaking rule. Here we choose `(e,s)`.

Merge 1: (e, s) → es

`n e w e s t </w>` becomes `n e w es t </w>`.

`w i d e s t </w>` becomes `w i d es t </w>`.

Merge 2: (es, t) → est

The pair `(es,t)` now has weighted count 9, producing `n e w est </w>` and `w i d est </w>`.

Later possible merges

`(l,o) → lo`, followed by `(lo,w) → low`. Frequent words may eventually become a single token.

Exercise 2: continue BPE

After the first two merges above:

1. Write the four transformed word representations.
2. Recount the pair `(l,o)`.
3. If `(l,o)` is merged, what new pair could form `low`?

4. WordPiece

WordPiece also learns a finite vocabulary of reusable subwords. Its training intuition differs from frequency-only BPE: a useful pair is one whose pieces occur together more strongly than expected from their individual frequencies.

$$\text{classroom score}(a,b) = \text{count}(a,b) \,/\, [\text{count}(a) \times \text{count}(b)]$$

The denominator penalizes pairs built from individually very common pieces. The original WordPiece method is described using improvement in a language-model likelihood. The formula above is a useful teaching approximation; individual libraries may implement details differently.

Worked score comparison

Pair `xy`: count(xy)=10, count(x)=100, count(y)=20

Pair `ab`: count(ab)=6, count(a)=10, count(b)=10

$$\text{score}(x,y) = 10 / (100 \times 20) = 0.005$$

$$\text{score}(a,b) = 6 / (10 \times 10) = 0.06$$

WordPiece's association score prefers `ab`, even though `xy` has the larger raw pair count.

4.1 Applying a fixed WordPiece vocabulary

At inference time, BERT-style WordPiece uses greedy longest matching from the start of the remaining word. The prefix `##` marks a piece that continues the current word.

```
Vocabulary fragment:
play, ##ing, ##er, un, ##able, able
```

```
playing  → play + ##ing
player   → play + ##er
unable   → un + ##able
```

The symbols `##` are a display convention, not a mathematical operation. If no valid decomposition exists, classic WordPiece may return an unknown token.

4.2 BPE and WordPiece compared

Aspect	BPE	WordPiece
Basic training choice	Most frequent adjacent pair	Pair with strong association / likelihood improvement
Simple classroom score	`count(a,b)`	`count(a,b)/(count(a)count(b))`

Encoding new text	Apply learned merges	Greedy longest vocabulary match
Common notation	Varies; may mark spaces or word boundaries	BERT uses `##` for continuation
Shared purpose	Represent open-ended text using a manageable reusable vocabulary	

Exercise 3: WordPiece matching

Vocabulary: `read`, `##ing`, `##er`, `un`, `##read`, `##able`. Segment:

1. `reader`
2. `reading`
3. `unreadable`

Which item may fail with this exact vocabulary, and why?

5. Embedding layers

5.1 A trainable lookup table

Token IDs are arbitrary integers. ID 27 is not “more meaningful” than ID 8. An embedding layer maps each ID to a trainable vector.

$$E \in \mathbb{R}^{V \times d}$$

$$\text{token ID } t_i \rightarrow x_i = E[t_i] \in \mathbb{R}^d$$

$$\text{IDs } [t_1, \ldots, t_n] \rightarrow X \in \mathbb{R}^{n \times d}$$

- *V*: number of vocabulary items
- *d*: embedding dimension or model width
- *n*: number of tokens in this sequence
- *E*: trainable embedding matrix

Lookup view

```
IDs = [27, 91, 8]
X = [
  E[27],
  E[91],
  E[8]
]
```

```
shape(X) = (3, d)
```

One-hot view

If $\mathbf{o}_t$ is a one-hot row vector of length V, then:

$$E[t] = \mathbf{o}_t E$$

Lookup is computationally preferable because it avoids constructing a mostly-zero vector.

5.2 Parameter and storage calculation

Example: V = 30,000 and d = 768

$$\text{Parameters} = 30{,}000 \times 768 = 23{,}040{,}000$$

$$\text{fp32 bytes} = 23{,}040{,}000 \times 4 = 92{,}160{,}000 \text{ bytes}$$

$$\approx 92.16 \text{ MB decimal} \approx 87.9 \text{ MiB}$$

fp16 or bfloat16 uses roughly 2 bytes per stored weight, so raw embedding weight storage is about 46.08 MB decimal.

Training requires additional memory for gradients, optimizer states, and activations. An untied output vocabulary projection may add another large matrix.

5.3 How embeddings learn

The training loss sends gradients through downstream layers into the selected embedding rows. If the input IDs are `[4, 9, 4]`, rows 4 and 9 receive gradients. Row 4 accumulates contributions from two positions:

$$\partial L/\partial E[4] = \partial L/\partial x_1 + \partial L/\partial x_3$$

$$\partial L/\partial E[9] = \partial L/\partial x_2$$

Static input embedding vs contextual representation

The input table contains one row for the token “bank.” Later Transformer layers create different hidden vectors for “river bank” and “bank loan” because surrounding context changes the representation.

5.4 Weight tying

A language model predicts a distribution over the vocabulary. Some models reuse the input embedding matrix in the output projection:

$$\text{logits} = hE^T$$

This is called **weight tying**. It reduces parameters and links the input and output token spaces. The dimensions must be compatible.

Exercise 4: shape and memory

A model uses V=50,000, d=512, batch size B=8, and sequence length n=128.

1. What is the shape of E?
2. How many embedding parameters are there?
3. What is the shape after embedding a batch of token IDs?
4. Approximately how much raw fp32 storage does E require?

6. Positional embeddings and encodings

6.1 Why order must be supplied

Consider two sentences containing the same three content words:

dog bites man
man bites dog

The token embeddings identify *dog*, *bites*, and *man*, but the same token has the same initial lookup vector at every position.

$$Q = XW_Q,\ K = XW_K,\ V = XW_V$$

$$\text{Attention}(X) = \text{softmax}(QK^T/\sqrt{d_k})V$$

Let Π be a permutation matrix that reorders the sequence rows. Without a positional signal:

$$\text{Attention}(\Pi X) = \Pi\ \text{Attention}(X)$$

The outputs follow the same reordering. This property is called **permutation equivariance**. It is useful for sets, but language requires order. Therefore the model needs explicit position information.

6.2 Input construction

$$\text{input}_i = \text{token_embedding}[\text{token}_i] + \text{position_embedding}[i] + \text{optional_segment_embedding}[\text{segment}_i]$$

Every vector has the same width *d*, so element-wise addition preserves the sequence matrix shape *n*×*d*.

6.3 Learned absolute positional embeddings

Learn a matrix $P \in \mathbb{R}^{L_{max} \times d}$, where *Lmax* is the maximum supported sequence length. Position *i* uses row P[i].

- **Advantage:** simple and flexible for trained positions.
- **Cost:** Lmax×d learned parameters.
- **Limitation:** unseen or longer positions are not naturally defined.

6.4 Sinusoidal positional encodings

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$

Each pair of dimensions uses a different frequency. Some dimensions change quickly, while others change slowly. Together they form a multi-scale signature for each position. Fixed sine/cosine identities also make relative offsets algebraically accessible.

Tiny numerical example

For *d*=4 and position 1:

$$PE(1) = [\sin(1), \cos(1), \sin(1/100), \cos(1/100)]$$

$\approx [0.8415, 0.5403, 0.0100, 0.99995]$

6.5 Modern alternatives

Some models use **relative position biases** or **rotary position embeddings (RoPE)**, which introduce position into attention scores or rotate query/key features. The mechanism changes, but the goal remains: make order and distance available to the model.

Exercise 5: position reasoning

1. If the same token appears at positions 2 and 10, which part is identical before addition?
2. Why must token and position embeddings have the same width?
3. What does a content-only attention layer do when all input rows are permuted?

7. Practical investigation

7.1 Inspect a real WordPiece tokenizer

Predict each split before running the program. Compare a common sentence, a typo, source code, a rare place name, and text outside the tokenizer's main training domain.

```
from transformers import AutoTokenizer, AutoModel

name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModel.from_pretrained(name)

samples = [
    "Tokenization is unbelievably useful.",
    "Tokenizashun is useful.",              # deliberate typo
    "vector<vector<int>> values;",          # source code
    "Thiruvananthapuram costs ₹1,23,456.",
]

for text in samples:
    ids = tokenizer.encode(text, add_special_tokens=True)
    pieces = tokenizer.convert_ids_to_tokens(ids)
    print("\nTEXT:", text)
    print("TOKENS:", pieces)
    print("COUNT:", len(ids))

print("\nEmbedding table:",
      tuple(model.get_input_embeddings().weight.shape))
```

Observation table

Input	Predicted pieces	Actual pieces	Token count	Observation
Common sentence				
Typo				
Source code				
Place name / number				
Your own example				

7.2 Verify embedding lookup

```
import torch

embedding = torch.nn.Embedding(
    num_embeddings=100,
    embedding_dim=4,
)

ids = torch.tensor([[27, 91, 8]])    # shape: (batch=1, sequence=3)
x = embedding(ids)                   # shape: (1, 3, 4)

print(embedding.weight.shape)          # torch.Size([100, 4])
print(x.shape)                         # torch.Size([1, 3, 4])

# Mathematically equivalent one-hot multiplication
one_hot = torch.nn.functional.one_hot(ids, num_classes=100).float()
print(torch.allclose(x, one_hot @ embedding.weight))  # True
```

Practical questions

1. Which sample produced the largest token count? Suggest a reason.
2. Did the typo increase token count or create an unknown token?
3. What do the two dimensions in `embedding.weight.shape` mean?
4. Why is direct lookup preferable to explicitly constructing one-hot vectors?

5. Try one sentence in another language. Is token usage comparable?

8. Food for thought

These questions do not always have one correct answer. A strong answer states a position and supports it with a mechanism or measurable consequence.

Cost and fairness

Should users of two languages pay the same token cost for equivalent meaning? What product or social consequences follow when token counts differ?

Robustness

When might byte-level tokenization be better than learned subwords despite longer sequences?

Meaning

Is a linguistically “ugly” split necessarily bad if downstream accuracy is high? What evidence would decide?

Changing domains

What happens when a tokenizer trained on prose is used for a new programming language, chemical notation, or legal citations?

Context windows

Is an 8,000-token context a fixed number of words? Which factors change the conversion?

Model quality

Could changing only the tokenizer alter accuracy while model size and training text remain fixed? Explain a causal path.

8.1 Design challenge

Propose a tokenizer

Choose one system: source-code completion, Indian-language translation, medical records, or social-media moderation. Propose:

1. the base symbols (characters, bytes, or another choice);
2. a vocabulary-size range;
3. the training corpus;
4. two evaluation metrics; and
5. one likely failure mode.

8.2 Take-home coding challenge

Implement a small BPE trainer. Represent words as lists of characters, count pairs using word frequencies, merge the maximum pair, and print the corpus after each of ten merges.

```
def pair_counts(corpus):
    counts = {}
    for symbols, frequency in corpus:
        for pair in zip(symbols, symbols[1:]):
            counts[pair] = counts.get(pair, 0) + frequency
    return counts

# TODO:
# 1. choose max(counts, key=counts.get)
# 2. merge adjacent occurrences in every word
# 3. repeat K times and store the merge order
# 4. test the learned units on a held-out word
```

9. Revision sheet

The complete story in six statements

1. Whole-word vocabularies are brittle; character/byte sequences can be long.
2. Subwords balance vocabulary size, sequence length, and unknown-word handling.
3. BPE greedily merges frequent adjacent symbols.
4. WordPiece favors pieces that associate strongly and uses longest matching.
5. An embedding layer is a trainable V×d lookup table.
6. Position information is necessary because content-only attention is permutation equivariant.

9.1 Essential equations

BPE: $\text{pair}^* = \arg\max \text{count}(a,b)$

WordPiece teaching score: $\text{count}(a,b) / [\text{count}(a)\text{count}(b)]$

Embedding lookup: $x_i = E[t_i]$, $E \in \mathbb{R}^{V \times d}$

Embedding parameters: $V \times d$

Transformer input: token embedding + position embedding

$PE(pos,2i) = \sin(pos/10000^{2i/d})$

$PE(pos,2i+1) = \cos(pos/10000^{2i/d})$

9.2 Glossary

Term	Meaning
Token	A discrete unit presented to a model: word, subword, character, byte, punctuation, or special symbol.
Vocabulary	The finite set of token types with assigned IDs.
Merge rule	An instruction that combines two adjacent BPE symbols.
Embedding	A learned dense vector associated with a discrete ID.
Embedding dimension d	The number of learned values in each token vector.
Contextual representation	A later hidden state that depends on surrounding tokens.

Positional embedding	A learned vector representing an absolute sequence position.
Positional encoding	A general position signal; it may be fixed rather than learned.
Permutation equivariance	Permuting inputs causes corresponding permutation of outputs.
Token fertility	How many tokens a tokenizer produces per word or unit of text.

9.3 Common misconceptions

A token is always a complete word.
WordPiece and BPE are identical because both produce subwords.
The tokenizer learns embedding vectors.
An embedding vector is a human-readable dictionary definition.
Token identity already contains sequence position.
A larger vocabulary is always better.

If you checked any statement above as true, revisit the corresponding section: every statement is a misconception.

9.4 One-minute self-test

1. Why not use one token per possible word?
2. If V doubles and d is fixed, what happens to embedding parameters?
3. Which embedding rows receive gradient for IDs [7, 2, 7]?
4. What information is missing from content-only self-attention?

10. Self-check answers

Attempt the exercises before reading this section. Open design questions can have multiple valid answers.

Quick check 1

1. Spell-checker: characters, bytes, or robust subwords reduce unknowns for errors.
2. Source code: byte-level or code-trained subwords handle novel identifiers and symbols.
3. Fixed 20-command classifier: whole commands or words may be sufficient because the domain is closed. The best choice depends on allowed variation.

Exercise 2

After `(e,s)→es` and `(es,t)→est`:

```
l o w </w>          ×5
l o w e r </w>      ×2
n e w est </w>      ×6
w i d est </w>      ×3
```

`(l,o)` still occurs 7 times. Merging it produces `lo`; the pair `(lo,w)` also occurs 7 times and can produce `low`.

Exercise 3

- `reader → read + ##er`
- `reading → read + ##ing`
- `unreadable` may fail with the exact listed vocabulary. After `un`, the continuation must match using continuation pieces. One possible required set is `un + ##read + ##able`, which is available here, so it succeeds. If `##read` were absent, having standalone `read` would not necessarily be enough in BERT-style notation.

Exercise 4

1. E has shape `(50,000, 512)`.
2. Parameters: 50,000×512 = 25,600,000.
3. Embedded batch shape: `(8, 128, 512)`.
4. fp32 storage: 25,600,000×4 = 102,400,000 bytes ≈102.4 MB decimal.

Exercise 5

1. The token embedding lookup is identical; the position component differs.
2. Element-wise addition requires vectors of equal width.
3. The outputs are permuted in the corresponding way; the layer does not infer an intrinsic order.

One-minute self-test

1. The possible-word set is open-ended, creating a huge vocabulary and unknown words.
2. Embedding parameters double because the count is V×d.
3. Rows 7 and 2; row 7 accumulates contributions from two positions.
4. Sequence order and relative/absolute position are missing.